

Narzędzia i Aplikacje Jakarta EE

Jakarta Contexts and Dependency Injection

Michał Wójcik

Wzorzec wstrzykiwania zależności

Wstrzykiwanie zależności - Klasa nie odpowiada za pozyskanie swoich zależności, zamiast tego zależności są do niej dostarczane z zewnątrz.

Konsekwencje:

- klasa nie musi wiedzieć, jak pozyskać zależność:
  - klasa nie jest uzależniona od sposobu pozyskania wymaganego komponentu (np. obiekt fabryki, builder, JNDI lookup, klasy narzędziowe typu `JPAUtils`)
  - klasa jest uzależniona wyłącznie od wymaganego komponentu;
- klasa nie musi podejmować decyzji, którą implementację zależności wykorzystać, jeśli jest ich wiele:
  - nie jest związana na stałe z jedną konkretną implementacją,
  - komponent dostarczający zależność wybiera implementację i ją wstrzykuje,
  - łatwość zamiany implementacji zależności wykorzystywanej przez klasę, bez konieczności modyfikowania samej klasy;
- klasa nie musi zarządzać (inaczej: przejmować się) cyklem życia swoich zależności:
  - ta odpowiedzialność spada na komponent dostarczający zależności,
  - upraszcza to projekt klasy wykorzystującej zależności.

Jak dostarczyć zależności

Wykorzystuje się trzy formy wstrzykiwania:

- przez argumenty konstruktora,
- przez settery,
- bezpośrednio do pól klasy.

Wstrzykiwanie przez konstruktor

```
public class LibraryService {

    private final UserNotifier notifier;

    public LibraryService(UserNotifier notifier) {
        this.notifier = notifier;
    }

}
```

Wstrzykiwanie przez parametry konstruktora:

- umożliwia tworzenie niezmiennych obiektów (ang. immutable):
  - pola opatrzone słowem kluczowym final;
- konstrukcja i wstrzyknięcie zależności w jednym kroku,
  - brak *konstruktora domyślnego*.

Wstrzykiwanie przez settery

```java
public class LibraryService {

    private UserNotifier notifier;

    public void setNotifier(UserNotifier notifier) {
        this.notifier = notifier;
    }

}
```

Wstrzykiwanie przez settery:

- obecny *konstruktor domyślny*,
- możliwość podmiany zależności po jej wstrzyknięciu:
  - przydatna tylko w pewnych zastosowaniach
- konieczny setter, który może nie być przydatny do niczego innego.

Wstrzykiwanie do pól

```
public class LibraryService {

    private UserNotifier notifier;

}
```

```
LibraryService service = new LibraryService();
Field field = service.getClass().getDeclaredField("notifier");
field.setAccessible(true);
UserNotifier notifier = new EmailUserNotifier();
field.set(service, notifier);
```

Wstrzykiwanie do pól:

- brak nadmiarowych metod,
- wstrzykiwanie wymaga użycia mechanizmów refleksji.

Bez wstrzykiwania - tworzenie zależności

```
public class LibraryService {

    private UserNotifier notifier;

    public LibraryService() {
        String filename = "config.properties";
        InputStream input = this.getClass().getClassLoader().getResourceAsStream(filename);

        Properties prop = new Properties();
        prop.load(input);

        String user = prop.getProperty("smtp.user");
        String password = prop.getProperty("smtp.password");
        notifier = new EmailUserNotifier("192.168.1.17", "991", user, password);
    }

}
```

Be wstrzykiwania - pozyskanie fabryki

```
public class LibraryService {

    private UserNotifier notifier;

    public LibraryService() {
        this.notifier = EmailUserNotifier.getInstance();
    }

}
```

```
public class LibraryService {

    private UserNotifier notifier;

    public LibraryService() {
        EmailNotifierFactory factory = EmailNotifierFactoryProvider.getFactory();
        this.notifier = factory.getNotifier();
    }

}
```

Dostarczania zależności

Dostarczanie zależności:

- pewien komponent aplikacji musi odpowiadać za tworzenie instancji zależności i przekazywanie ich do wymagających tego obiektów,
- dopiero po dostarczeniu wszystkich zależności, obiekt jest gotowy do użycia:
  - dopiero wtedy można przekazać do niego sterowanie;
- komponent tworzący instancje obiektów powinien dostarczyć do nich zależności, zanim przekaże je dalej lub zacznie wykorzystywać.

Odwrócenie sterowania

Tradycyjny przepływ sterowania:

- komponenty aplikacji kontrolują sterowanie,
- w kluczowych momentach komponenty aplikacji wywołują zewnętrzne biblioteki, aby zrealizować swoją funkcjonalność,
- np. tradycyjne aplikacje okienkowe z pętlą obsługi zdarzeń.

Odwrócenie sterowania:

- przepływ sterowania kontrolowany przez kontener wykonawczy aplikacji,
- w kluczowych etapach kontener wykonawczy wywołuje komponenty aplikacji, np.:
  - serwer Tomcat przyjmuje żądanie HTTP, przetwarza je i w pewnym momencie przekazuje sterowanie do jednego z servletów,
  - wybór servletu do wywołania wynika z konfiguracji aplikacji,
  - gdy Servlet kończy pracę, sterowanie wraca do serwera Tomcat, który przygotowuje odpowiedź HTTP i wysyła ją do przeglądarki.

DI vs IoC

Wstrzykiwanie zależności (ang. dependency injection) oraz **Odwrócenie sterowania** (ang. inversion of control) to dwa **odrębne wzorce**!

Wstrzykiwanie zależności (DI):

- prosty wzorzec projektowy,
- zależności nie są pozyskiwane przez obiekty, lecz do nich dostarczane,
- wstrzykiwanie przez konstruktory, settery lub bezpośrednio do pól.

Odwrócenie sterowania (IoC):

- wzorzec architektoniczny, opisujący przepływ sterowania w aplikacji,
- kontener wykonawczy aplikacji kontroluje sterowanie
- w kluczowych etapach kontener wykonawczy wywołuje komponenty aplikacji.

DI i IoC

DI i IoC:

- każdy z mechanizmów może być zastosowany osobno,
- często wykorzystywane są razem:
  - kontener wykonawczy zarządza cyklem życia komponentów składających się na aplikację,
  - po utworzeniu instancji obiektu realizuje wstrzykiwanie zależności,
  - gdy wszystkie zależności są dostarczone do obiektu, możliwe jest przekazanie do niego sterowania,
  - w tym podejściu kontener IoC wykorzystuje DI.

Java i wstrzykiwanie zależności

Specyfikacja JSR 330: *Dependency Injection for Java*:

- definiuje zestaw podstawowych interfejsów pozwalających na realizację wstrzykiwania zależności,
- wiele frameworków i bibliotek, które implementowały wstrzykiwanie zależności przed specyfikacją JSR 330, obecnie jest z nią zgodna:
  - Java EE,
  - framework Spring,
  - Play Framework,
  - framework Tapestry,
  - biblioteka Google Guice,
  - ...
- obejmuje trzy formy wstrzykiwania: przez parametry konstruktora, przez settery oraz bezpośrednio do pól.

Specyfikacja CDI

Jakarta Contexts and Dependency Injection (CDI):

- wcześniej: Contexts and Dependency Injection for Java EE,
- wersje:
  - JSR 299 - CDI 1.0 (Java EE 6),
  - JSR 346 - CDI 1.1 (Java EE 7),
  - JSR 365 - CDI 2.0 (Java EE 8),
  - CDI 3.0 (Jakarta EE 9),
  - CDI 4.0 (Jakarta EE 10);
- zgodne z JSR 330, uzupełniają ją o dodatkowe elementy,
- platforma docelowa: Java EE:
  - obowiązkowy element w *Web Profile* i *Full Profile*;
- implementacje możliwe do wykorzystania również poza Javą EE,
- referencyjna implementacja: biblioteka Weld.

CDI - konfiguracja projektu

CDI definiuje dwa rodzaje archiwów aplikacji:

- *explicit* - archiwum zawierające plik `beans.xml` (`WEB-INF/beans.xml`),
- *implicit* - archiwum pozbawione pliku `beans.xml`, obsługuje jedynie klasy, o jawnie zdefiniowanym zasięgu.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
                           https://jakarta.ee/xml/ns/jakartaee/beans_4_0.xsd"
       version="4.0" bean-discovery-mode="all">
</beans>
```

CDI - zarządzane klasy

Kontener CDI zarządza klasami, które spełniają poniższe warunki:

- nie są klasami wewnętrznymi,
- są klasami konkretnymi (nie są abstrakcyjne) lub są oznaczone adnotacją `@Decorator`,
- nie implementują `jakarta.enterprise.inject.spi.Extension`,
- posiadają konstruktor domyślny lub konstruktor, którego wszystkie argumenty mogą zostać wstrzyknięte przez CDI (oznaczony adnotacją `@Inject`),
- dla archiwów *implicit* lub z ustawionym `bean-discovery-mode="annotated"`: są opatrzone adnotacją zasięgu.

CDI - podstawowe wstrzykiwanie

Realizacja podstawowego wstrzykiwania poprzez: `@Inject` - wskazuje miejsce wstrzyknięcia zależności (JSR 330),

- bez dodatkowych adnotacji wstrzykuje na podstawie typu,
- dla wstrzykiwania po typie nie są wymagane adnotacje na klasie wstrzykiwanej (`bean-discovery-mode="all"`),
- może być wykorzystywane do wstrzykiwania beanów EJB.

Wstrzykiwanie przez konstruktor - CDI

```
public class LibraryService {

    private final UserNotifier notifier;

    @Inject
    public LibraryService(UserNotifier notifier) {
        this.notifier = notifier;
    }

}
```

Wstrzykiwanie przez settery - CDI

```
public class LibraryService {

    private UserNotifier notifier;

    @Inject
    public void setNotifier(UserNotifier notifier) {
        this.notifier = notifier;
    }

}
```

Wstrzykiwanie do pól - CDI

```
public class LibraryService {

    @Inject
    private UserNotifier notifier;

}
```

Zasięgi

Zasięgi:

- kontenery zależności definiują zazwyczaj zasięgi, w których mogą znajdować się wstrzykiwane obiekty,
- obiekty o przypisanym zasięgu, są dostępne tylko w jego obrębie:
  - gdy sterowanie aplikacji znajduje się poza danym zasięgiem, obiekty do niego przypisane nie są dostępne,
  - gdy zasięg jest zamykany, niszczone są obiekty, które się w nim znajdują.

Zasięgi - JSR 330

JSR 330 - ze względu na swój uniwersalny charakter, definiuje jedynie dwa zasięgi:

- domyślny – każde wstrzyknięcie tworzy nową instancję,
- `@Singleton` – pojedyncza instancja wykorzystywana w całej aplikacji.

Dodatkowe zasięgi mogą być definiowane przez implementacje JSR 330 w zależności od przeznaczenia, np.:

- dla aplikacji internetowych: zasięg zapytania HTTP, zasięg sesji,
- dla aplikacji okienkowych: zasięg wątku.

Zasięgi - CDI

CDI - jako specyfikacja ukierunkowana na aplikacje internetowe Java EE, definiuje zasięgi:

- `@Dependent` - odpowiada domyślnemu zasięgowi z JSR 330,
- `@RequestScoped` - zasięg pojedynczego żądania HTTP,
- `@ConversationScoped` - zasięg konwersacji, może obejmować kilka kolejnych zapytań HTTP (np. wieloetapowy formularz),
- `@SessionScoped` - zasięg sesji, aktywny, dopóki sesja użytkownika nie wygaśnie,
- `@ApplicationScoped` - zasięg aplikacji, aktywny od momentu wdrożenia do momentu usunięcia z serwera,
- `@Singleton` - zgodny z JSR 330.

Dodatkowo dla JSF:

- `@ViewScoped` - zasięg wyświetlania pojedynczego widoku JSF.

Dodatkowo dla JTA:

- `@TransactionScoped` - zasięg transakcji JTA.

Widoczność zasięgów CDI

Gdzie dostępne zą zasięgi:

- `@RequestScoped`, `@ConversationScoped`, `@SessionScoped`, `@ApplicationScoped`:
  - każde żądanie servletu,
- `@RequestScoped`, `@ApplicationScoped`:
  - wywołania zdalnych metod EJB,
  - wywołania asynchronicznych metod EJB,
  - EJB timeout,
  - dostarczanie wiadomości do MD-EJB,
  - wywołania usług sieciowych;
- `@ApplicationScoped`:
  - wywołania `@PostConstruct` i `@PreDestroy` beanów;
- `@TransactionScoped`:
  - aktywna transakcja JAT.

Zasięgi - Dependent

Zasięg `@Dependent`:

- domyślny zasięg dla beanów, którym nie przypisano żadnej adnotacji zasięgu,
- można jawnie wskazać ten zasięg adnotacją `@jakarta.enterprise.context.Dependent`,
- tworzy nową instancję beana przy każdym wstrzyknięciu,
- instancja jest niszczona, kiedy obiekt, do którego została wstrzyknięta, jest niszczony.

Zasięgi - RequestScoped

Zasięg `@RequestScoped`:

- adnotacja `@jakarta.enterprise.context.RequestScoped`,
- obiekt istnieje w czasie obsługi żądania HTTP,
- gdy żądanie się kończy, instancje są niszczone.

Zasięgi - ConversationScoped

Zasięg `@ConversationScoped`:

- adnotacja `@jakarta.enterprise.context.ConversationScoped`,
- umożliwia prostą implementację wieloetapowych procesów,
- do zarządzania stanem konwersacji służy obiekt typu `Conversation`, który wstrzykujemy do beana,
- dwa stany konwersacji:
  - chwilowy - odpowiada `@RequestScoped`, domyślny stan po utworzeniu beana,
  - długotrwały - bean przejdzie w ten stan po wywołaniu `conversation.begin()`;
- bean w długotrwałej konwersacji jest niszczony, gdy zakończymy konwersację `conversation.end()` i zakończy się żądanie HTTP.

Zasięgi - SessionScoped

Zasięg `@SessionScoped`:

- adnotacja `@jakarta.enterprise.context.SessionScoped`,
- obejmuje sesję HTTP,
- możne przechowywać dane związane z jednym użytkownikiem,
- bean niszczony, gdy niszczona jest sesja.

Zasięgi - ApplicationScoped

Zasięg `@ApplicationScoped`:

- adnotacja `@jakarta.enterprise.context.ApplicationScoped`,
- obiekt tworzony raz na cały czas działania aplikacji,
- może zawierać obiekty współdzielone pomiędzy różnymi użytkownikami.

Zasięgi - Singleton

Zasięg `@Singleton`:

- traktuje beana jako Singleton,
- adnotacja `@jakarta.inject.Singleton` (uwaga na pakiet),
- Weld: beany, do których wstrzykniemy Singleton, otrzymają bezpośrednią referencję - problem z serializacją, który należy samodzielnie rozwiązać poprzez:
  - `writeResolve()`, `readReplace()`,
  - nietrwałe referencje (`transient`),
  - programistyczne odpytywanie kontenera CDI;
- dokumentacja Weld zaleca użycie beanów o zasięgu `@ApplicationScoped` zamiast Singletonów.

Zasięgi – ViewScoped

Zasięg `@ViewScoped`:

- zasięg związany z widokiem JSF,
- od Java EE 7 standardowy zasięg CDI,
- `jakarta.faces.view.ViewScoped`,
- nie jest częścią specyfikacji CDI,
- jest częścią specyfikacji JSF.

Zasięgi - TransactionScoped

Zasięg `@TransactionScoped`:

- zasięg związany z transakcją JTA,
- od Java EE 7 standardowy zasięg CDI,
- `javax.transaction.TransactionScoped`,
- nie jest częścią specyfikacji CDI,
- jest częścią specyfikacji JTA.

CDI a EL

Dostęp do beanów CDI z poziomu EL (Expression Language) jest dostępny po ich nazwaniu.

```
@ViewScoped
@Named
public class BookView {

    @Getter
    private List<Book> books = new ArrayList<>();

}
```

```
${bookView.books}
```

Pomocne źródła

Specyfikacja:

- https://jakarta.ee/specifications/cdi/4.1/jakarta-cdi-spec-4.1

Uzupełnienie informacji:

- https://jakarta.ee/learn/docs/jakartaee-tutorial/current/cdi/cdi-basic/cdi-basic.html
- https://www.baeldung.com/java-ee-cdi